CHAPTER – III

CONTROL STRUCTURE AND LOOPING STATEMENTS

CONTROL STRUTURES:

Control structures are the decision making and looping statements in java. It checks the condition and executes the code accordingly.

TYPES:

- Simple If condition
- If else condition
- Else if ladder
- Nested if

SIMPLE IF :

It is used to execute a block of code only when the condition is true.

SYNTAX:

```
if(condition) {
        //statements
```

EXAMPLE:

```
public class Age{
    public static void main(String [] args){
     int age = 20;
        if (age >= 18) {
        System.out.println("You are an adult.");
        }
    }
}
```

ELSE IF :

The if-else statement is used to execute one block of code when a condition is true and another block when the condition is false.

SYNTAX:

```
if (condition) {
        //statements
}
else {
        //statements
}
```

EXAMPLE:

```
public class Example {
        public static void main(String[] args) {
                int age=16;
                if(age>=18){
                        System.out.println(“Eligible to vote”);
                }
                else {
                        System.out.println(“Not eligible”);
                }
        }
}
```

ELSE IF LADDER:

An else-if ladder is used when there are multiple conditions to check. Java evaluates the conditions from top to bottom and executes the block of the first condition that is true.

SYNTAX:

```
if(condition) {
        //statements
}
else if(condition) {
        //statements
}
else if(condition) {
        //statements
}
else {
        //statements
}
```

EXAMPLE:

```
public class Example {
        public static void main(String [] args) {
                int marks = 50;
                if (marks>=40){
                        System.out.println(“Grade A”);
                else if (marks>=30){
                        System.out.println(“Grade B”);
                else if(marks>=20){
                        System.out.println(“Grade C”);
                else{
                        System.out.println(“Fail”);
                }
```

```
    }
}
```

NESTED IF :

A nested if statement means placing one if statement inside another if statement. It is used when a second condition should be checked only if the first condition is true.

SYNTAX:

```
if (condition1) {
        // executes if condition1 is true
  if (condition2) {
        // executes if both condition1 and condition2 are true
   }
 }
```

EXAMPLE:

```
public class NestedIfExample1 {
        public static void main(String[] args) {
                int age 20;
                String nationality "Indian";
                if (age 18) {
                        if (nationality.equals("Indian")) (
                                System.out.println("You eligible to vote in India.");
                        } else {
                        System.out.println("You are not eligible to vote in India.");
                        }
                else {
```

```
            System.out.println("You must be at least 18 years old to vote.");
        }
    }
}
```

SWITCH CASE STATEMENTS:

A switch case statement is used to select one block of code from multiple options based on the value of a variable or expression.

SYNTAX:

```
switch (variable) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
        // statements
    }
```

EXAMPLE:

```
public class Day {
    public static void main(String[] args) {
    int day = 3;
    switch(day) {
      case 1:
        System.out.println("Monday");
        break;
      case 2:
        System.out.println("Tuesday");
```

```
        break;
          case 3:
           System.out.println("Wednesday");
             break;
          default:
             System.out.println("Invalid Day");
               }
        }
}
```

FOR LOOP:

A for loop is used to execute a block of code repeatedly for a specific number of times.

SYNTAX:

```
for (initialization; condition; update) {
   // statements
 }
```

EXAMPLE:

```
public class Example {
        public static void main(String [] args) {
                for(int i=1; i<=5; i++) {
                        System.out.println("The number:");
                }
        }
}
```

NESTED FOR LOOP:

A nested for loop is a loop inside another loop. The inner loop executes completely for each iteration of the outer loop.

EXAMPLE:

```
public class AlphabetPattern {
	public static void main(String[] args) {
		int n = 5;
		for (int i = 1; i <= n; i++) {
			char ch = 'A';
			for (int j = 1; j <= i; j++) {
				System.out.print(ch + " ");
				ch++;
			}
			System.out.println();
		}
	}
}
```

EXAMPLE 2 (PATTERN PRINTING):

```
public class InvPyr {
	public static void main(String[] args) {
		int n = 5;
		for (int i = n; i >= 1; i--) {
			for (int j = 1; j <= n - i; j++) {
				System.out.print(" ");
			}
			for (int k = 1; k <= i; k++) {
```

```
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```

EXAMPLE 3(HOLLOW PRINTING):

```
public class InvRightTri {
    public static void main(String[] args) {
        int rows=5;
        int colu=5;
        for(int i=1; i<=rows; i++) {
            for(int j=1; j<=colu; j++) {
                if(i==1||j==1||i+j==6)
                    System.out.print("*");
                else
                    System.out.print(" ");
            }
            System.out.println();
        }
    }
}
```

WHILE LOOP:

A while loop is used to repeatedly execute a block of code as long as a condition is true.

SYNTAX:

```
while (condition) {
   // statements
}
```

EXAMPLE:

```
public class EvenNum {
        public static void main(String[] args) {
        int i = 1;
                while(i <= 5) {
                System.out.println(i);
                i++;
                }
        }
}
```

DO WHILE LOOP:

A do-while loop is used to execute a block of code at least once, and then repeatedly execute it as long as the condition is true.

SYNTAX:

```
do {
        // statements
}
while (condition);
```

EXAMPLE:

```
public class DoWhileExample {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println( i)

            ; i++;
        } while (i <= 5);
    }
}
```

ENHANCED FOR LOOP:

The Enhanced For Loop (or For-Each Loop) is used to traverse elements of an array or collection without using an index.

SYNTAX:

```
for (dataType variable : arrayName) {
    // statements
}
```

EXAMPLE:

```
public class Example {
    public static void main(String[] args) {
        int[] marks = {80, 85, 90, 95};
        for(int mark : marks) {
            System.out.println(mark);
        }
    }
}
```

BREAK STATEMENT:

The break statement is used to immediately terminate a loop or switch statement and transfer control to the statement following it.

EXAMPLE:

```
public class BreakExample1 {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                break
            }
            System.out.println(i);
        }
    }
}
```

CONTINUE STATEMENT:

The continue statement is used to skip the current iteration of a loop and continue with the next iteration.

EXAMPLE:

```
Public class Example{
    Public static void main(String [] args) {
        for(int i = 1; i <= 5; i++) {
            if(i == 3) {
                continue;
            }
            System.out.println(i);
        }
    }
}
```